Assignment 6

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question

Write a C program to represent and analyze two real-world networks using graph data structures, where one graph is sparse and the other is dense.Let the number of vertices be $n = 100$. For the sparse graph, take the number of edges as $e_s = 120$, such that

$$e_s \ll \frac{n(n-1)}{2}$$

and generate the edges randomly. Represent this sparse graph using an adjacency list for both unweighted edges (showing only connections) and weighted edges (assigning a random cost or distance to each edge).

For the dense graph, take the number of edges as $e_d = 4500$, such that

$$e_d \approx \frac{n(n-1)}{2}$$

and generate the edges randomly. Represent this dense graph using an adjacency matrix for both unweighted and weighted edges. The program should automatically generate random vertex pairs and edge weights, construct the corresponding graph representations, and display the adjacency list and adjacency matrix.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 100
#define E_SPARSE 120
#define E_DENSE 4500

typedef struct Node {
    int dest;
    int weight;
    struct Node* next;
} Node;

int sparse_edges[N][N] = {0};
int dense_edges[N][N] = {0};

```

```
Node* sparse_adj_list_unweighted[N] = {NULL};
Node* sparse_adj_list_weighted[N] = {NULL};

int dense_adj_matrix_unweighted[N][N] = {0};
int dense_adj_matrix_weighted[N][N] = {0};

void addEdgeList(Node* adj_list[], int src, int dest, int weight) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->dest = dest;
    newNode->weight = weight;
    newNode->next = adj_list[src];
    adj_list[src] = newNode;

    newNode = (Node*)malloc(sizeof(Node));
    newNode->dest = src;
    newNode->weight = weight;
    newNode->next = adj_list[dest];
    adj_list[dest] = newNode;
}

int main() {
    srand(time(NULL));
    int u, v, w, count;

    count = 0;
    while (count < E_SPARSE) {
        u = rand() % N;
        v = rand() % N;
        if (u != v && !sparse_edges[u][v]) {
            w = (rand() % 50) + 1;
            sparse_edges[u][v] = 1;
            sparse_edges[v][u] = 1;
            addEdgeList(sparse_adj_list_unweighted, u, v, 1);
            addEdgeList(sparse_adj_list_weighted, u, v, w);
            count++;
        }
    }

    count = 0;
    while (count < E_DENSE) {
        u = rand() % N;
        v = rand() % N;
        if (u != v && !dense_edges[u][v]) {
            w = (rand() % 50) + 1;
            dense_edges[u][v] = 1;
            dense_edges[v][u] = 1;

            dense_adj_matrix_unweighted[u][v] = 1;
            dense_adj_matrix_unweighted[v][u] = 1;

            dense_adj_matrix_weighted[u][v] = w;
```

```c
            dense_adj_matrix_weighted[v][u] = w;
            count++;
        }
    }

    printf("Graphs Generated Successfully.\n");
    printf("\nSparse Graph Adjacency List (Weighted) - First 5 Vertices:\n");
    for (int i = 0; i < 5; i++) {
        printf("Vertex %d: ", i);
        Node* temp = sparse_adj_list_weighted[i];
        while (temp) {
            printf("-> %d(w:%d) ", temp->dest, temp->weight);
            temp = temp->next;
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> gcc .\Ques1.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> ./a.exe
Graphs Generated Successfully.

Sparse Graph Adjacency List (Weighted) - First 5 Vertices:
Vertex 0: -> 80(w:1) -> 29(w:15) -> 7(w:45) -> 57(w:25)
Vertex 1:
Vertex 2: -> 43(w:1) -> 77(w:21)
Vertex 3: -> 25(w:5) -> 48(w:24) -> 57(w:22) -> 24(w:44) -> 88(w:43) -> 71(w:6)
Vertex 4: -> 9(w:49)
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6>
```

Figure 1: Output for Problem 1.

Problem 2

Question

Write a C program to connect 6 cities (C1-C6) with minimum cost using Prim's Algorithm and a Min-Priority Queue. The cost of connecting each pair of cities (in lakhs) is: $(C1, C2) = 3$, $(C1, C3) = 1$, $(C1, C4) = 6$, $(C2, C3) = 5$, $(C2, C5) = 3$, $(C3, C4) = 5$, $(C3, C6) = 4$, $(C4, C6) = 2$, $(C5, C6) = 6$.

Your program should:

1. Represent the graph using an adjacency list.
2. Implement Prim's Algorithm starting from vertex C1 using a Min-Priority Queue.
3. Display the edges in the Minimum Spanning Tree (MST), the total minimum cost, and the order of vertices added.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define V 6

typedef struct AdjListNode {
    int dest;
    int weight;
    struct AdjListNode* next;
} AdjListNode;

typedef struct MinHeapNode {
    int v;
    int key;
} MinHeapNode;

typedef struct MinHeap {
    int size;
    int capacity;
    int *pos;
    MinHeapNode **array;
} MinHeap;

AdjListNode* newAdjListNode(int dest, int weight) {
    AdjListNode* newNode = (AdjListNode*)malloc(sizeof(AdjListNode));
    newNode->dest = dest;
    newNode->weight = weight;
    newNode->next = NULL;
    return newNode;
}

```

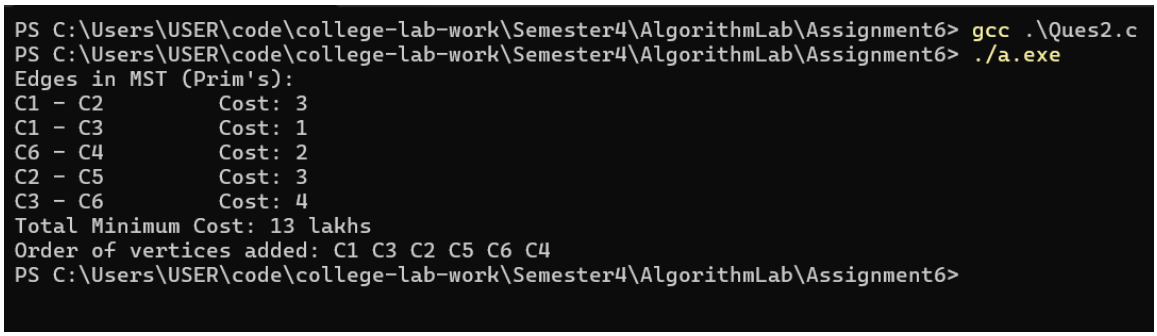
```c
void addEdge(AdjListNode* graph[], int src, int dest, int weight) {
    AdjListNode* newNode = newAdjListNode(dest, weight);
    newNode->next = graph[src];
    graph[src] = newNode;

    newNode = newAdjListNode(src, weight);
    newNode->next = graph[dest];
    graph[dest] = newNode;
}

MinHeapNode* newMinHeapNode(int v, int key) {
    MinHeapNode* minHeapNode = (MinHeapNode*)malloc(sizeof(MinHeapNode));
    minHeapNode->v = v;
    minHeapNode->key = key;
    return minHeapNode;
}

MinHeap* createMinHeap(int capacity) {
    MinHeap* minHeap = (MinHeap*)malloc(sizeof(MinHeap));
    minHeap->pos = (int*)malloc(capacity * sizeof(int));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (MinHeapNode**)malloc(capacity * sizeof(MinHeapNode*));
    return minHeap;
}

void swapMinHeapNode(MinHeapNode** a, MinHeapNode** b) {
    MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}

void minHeapify(MinHeap* minHeap, int idx) {
    int smallest, left, right;
    smallest = idx;
    left = 2 * idx + 1;
    right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->key < minHeap->array[smallest]->key)
        smallest = left;
    if (right < minHeap->size && minHeap->array[right]->key < minHeap->array[smallest]->key)
        smallest = right;

    if (smallest != idx) {
        MinHeapNode* smallestNode = minHeap->array[smallest];
        MinHeapNode* idxNode = minHeap->array[idx];
        minHeap->pos[smallestNode->v] = idx;
```

```
        minHeap->pos[idxNode->v] = smallest;
        swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

MinHeapNode* extractMin(MinHeap* minHeap) {
    if (minHeap->size == 0) return NULL;
    MinHeapNode* root = minHeap->array[0];
    MinHeapNode* lastNode = minHeap->array[minHeap->size - 1];
    minHeap->array[0] = lastNode;
    minHeap->pos[root->v] = minHeap->size - 1;
    minHeap->pos[lastNode->v] = 0;
    --minHeap->size;
    minHeapify(minHeap, 0);
    return root;
}

void decreaseKey(MinHeap* minHeap, int v, int key) {
    int i = minHeap->pos[v];
    minHeap->array[i]->key = key;
    while (i && minHeap->array[i]->key < minHeap->array[(i - 1) / 2]->key) {
        minHeap->pos[minHeap->array[i]->v] = (i - 1) / 2;
        minHeap->pos[minHeap->array[(i - 1) / 2]->v] = i;
        swapMinHeapNode(&minHeap->array[i], &minHeap->array[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

int isInMinHeap(MinHeap* minHeap, int v) {
    return minHeap->pos[v] < minHeap->size;
}

void primMST(AdjListNode* graph[]) {
    int parent[V];
    int key[V];
    MinHeap* minHeap = createMinHeap(V);

    for (int v = 1; v < V; ++v) {
        parent[v] = -1;
        key[v] = INT_MAX;
        minHeap->array[v] = newMinHeapNode(v, key[v]);
        minHeap->pos[v] = v;
    }

    key[0] = 0;
    minHeap->array[0] = newMinHeapNode(0, key[0]);
    minHeap->pos[0] = 0;
```

```c
    minHeap->size = V;

    int add_order[V];
    int order_idx = 0;

    while (minHeap->size != 0) {
        MinHeapNode* minHeapNode = extractMin(minHeap);
        int u = minHeapNode->v;
        add_order[order_idx++] = u;

        AdjListNode* pCrawl = graph[u];
        while (pCrawl != NULL) {
            int v = pCrawl->dest;
            if (isInMinHeap(minHeap, v) && pCrawl->weight < key[v]) {
                key[v] = pCrawl->weight;
                parent[v] = u;
                decreaseKey(minHeap, v, key[v]);
            }
            pCrawl = pCrawl->next;
        }
    }

    int total_cost = 0;
    printf("Edges in MST (Prim's):\n");
    for (int i = 1; i < V; ++i) {
        printf("C%d - C%d \tCost: %d\n", parent[i] + 1, i + 1, key[i
    ]);
        total_cost += key[i];
    }
    printf("Total Minimum Cost: %d lakhs\n", total_cost);

    printf("Order of vertices added: ");
    for(int i = 0; i < V; i++) printf("C%d ", add_order[i] + 1);
    printf("\n");
}

int main() {
    AdjListNode* graph[V] = {NULL};

    addEdge(graph, 0, 1, 3); // C1-C2
    addEdge(graph, 0, 2, 1); // C1-C3
    addEdge(graph, 0, 3, 6); // C1-C4
    addEdge(graph, 1, 2, 5); // C2-C3
    addEdge(graph, 1, 4, 3); // C2-C5
    addEdge(graph, 2, 3, 5); // C3-C4
    addEdge(graph, 2, 5, 4); // C3-C6
    addEdge(graph, 3, 5, 2); // C4-C6
    addEdge(graph, 4, 5, 6); // C5-C6

    primMST(graph);
    return 0;
```

```
}
```

Output



```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> gcc .\Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> ./a.exe
Edges in MST (Prim's):
C1 - C2         Cost: 3
C1 - C3         Cost: 1
C6 - C4         Cost: 2
C2 - C5         Cost: 3
C3 - C6         Cost: 4
Total Minimum Cost: 13 lakhs
Order of vertices added: C1 C3 C2 C5 C6 C4
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6>
```

Figure 2: Output for Problem 2.

Problem 3

Question

Write a C program to connect 6 cities (C1-C6) with minimum cost using Kruskal's Algorithm. The cost of connecting each pair of cities (in lakhs) is: $(C1, C2) = 3$, $(C1, C3) = 1$, $(C1, C4) = 6$, $(C2, C3) = 5$, $(C2, C5) = 3$, $(C3, C4) = 5$, $(C3, C6) = 4$, $(C4, C6) = 2$, $(C5, C6) = 6$.

Your program should:

1. Represent the graph using an edge list.
2. Sort all edges in ascending order and construct the Minimum Spanning Tree (MST) using Union-Find (Disjoint Set) to prevent cycles.
3. Display the edges included in the MST, the total minimum cost, and the order in which edges are added.

Code

```c
#include <stdio.h>
#include <stdlib.h>

#define V_COUNT 6
#define E_COUNT 9

typedef struct Edge {
    int u, v, weight;
} Edge;

typedef struct subset {
    int parent;
    int rank;
} subset;

int find(subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

void Union(subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
```

```
    }
}

int compareEdges(const void* a, const void* b) {
    Edge* a1 = (Edge*)a;
    Edge* b1 = (Edge*)b;
    return a1->weight - b1->weight;
}

void kruskalMST(Edge edges[]) {
    Edge result[V_COUNT];
    int e = 0;
    int i = 0;

    qsort(edges, E_COUNT, sizeof(edges[0]), compareEdges);

    subset* subsets = (subset*)malloc(V_COUNT * sizeof(subset));
    for (int v = 0; v < V_COUNT; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    while (e < V_COUNT - 1 && i < E_COUNT) {
        Edge next_edge = edges[i++];
        int x = find(subsets, next_edge.u);
        int y = find(subsets, next_edge.v);

        if (x != y) {
            result[e++] = next_edge;
            Union(subsets, x, y);
        }
    }

    int minimumCost = 0;
    printf("Edges included in the MST (listed in the order they were added):\n");
    for (i = 0; i < e; ++i) {
        printf("Step %d: Edge C%d - C%d \tCost: %d\n", i + 1, result[i].u + 1, result[i].v + 1, result[i].weight);
        minimumCost += result[i].weight;
    }

    printf("\nTotal Minimum Cost: %d lakhs\n", minimumCost);

    printf("Order of edges added: ");
    for (i = 0; i < e; ++i) {
        printf("(C%d, C%d) ", result[i].u + 1, result[i].v + 1);
    }
    printf("\n");
}

```

```c
int main() {
    Edge edges[E_COUNT] = {
        {0, 1, 3}, {0, 2, 1}, {0, 3, 6},
        {1, 2, 5}, {1, 4, 3}, {2, 3, 5},
        {2, 5, 4}, {3, 5, 2}, {4, 5, 6}
    };

    kruskalMST(edges);
    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> gcc .\Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> ./a.exe
Edges included in the MST (listed in the order they were added):
Step 1: Edge C1 - C3    Cost: 1
Step 2: Edge C4 - C6    Cost: 2
Step 3: Edge C1 - C2    Cost: 3
Step 4: Edge C2 - C5    Cost: 3
Step 5: Edge C3 - C6    Cost: 4

Total Minimum Cost: 13 lakhs
Order of edges added: (C1, C3) (C4, C6) (C1, C2) (C2, C5) (C3, C6)
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6>
```

Figure 3: Output for Problem 3.

Problem 4

Question

Write a C program to compute the Minimum Spanning Tree (MST) of a large-scale smart city power distribution network consisting of two independent weighted graphs $G_1(V_1, E_1)$ and $G_2(V_2, E_2)$, interconnected by a single high-capacity transmission edge having the maximum weight among all edges. This high-weight edge must always be included in the final MST. Apply Kruskal's algorithm on graph G_1 considering it as a sparse graph, and apply Prim's algorithm on graph G_2 considering it as a dense graph. After computing the MSTs of both graphs separately, merge them using the mandatory high-weight edge to construct the global MST of the combined network. The program should take the number of vertices, edges, and weighted edge lists as input, compute the final MST, display the selected edges, and output the total cost of the spanning tree.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct Edge { int u, v, weight; } Edge;
typedef struct subset { int parent; int rank; } subset;

int find(subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}

void Union(subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);
    if (subsets[xroot].rank < subsets[yroot].rank) subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank) subsets[yroot].parent = xroot;
    else { subsets[yroot].parent = xroot; subsets[xroot].rank++; }
}

int compareEdges(const void* a, const void* b) {
    return ((Edge*)a)->weight - ((Edge*)b)->weight;
}

int kruskalMST(Edge edges[], int V, int E) {
    qsort(edges, E, sizeof(edges[0]), compareEdges);
    subset* subsets = (subset*)malloc(V * sizeof(subset));
    for (int v = 0; v < V; ++v) { subsets[v].parent = v; subsets[v].rank = 0; }

```

```c
    int mst_cost = 0;
    int e = 0, i = 0;
    printf("\n--- G1 MST Edges (Kruskal) ---\n");
    while (e < V - 1 && i < E) {
        Edge next_edge = edges[i++];
        int x = find(subsets, next_edge.u);
        int y = find(subsets, next_edge.v);
        if (x != y) {
            printf("%d - %d \tCost: %d\n", next_edge.u, next_edge.v, next_edge.weight);
            mst_cost += next_edge.weight;
            Union(subsets, x, y);
            e++;
        }
    }
    return mst_cost;
}

int minKey(int key[], int mstSet[], int V) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}

int primMST(int** graph, int V) {
    int* parent = (int*)malloc(V * sizeof(int));
    int* key = (int*)malloc(V * sizeof(int));
    int* mstSet = (int*)malloc(V * sizeof(int));

    for (int i = 0; i < V; i++) { key[i] = INT_MAX; mstSet[i] = 0; }
    key[0] = 0; parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u; key[v] = graph[u][v];
            }
    }

    int mst_cost = 0;
    printf("\n--- G2 MST Edges (Prim) ---\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \tCost: %d\n", parent[i], i, graph[i][parent[i]]);
        mst_cost += graph[i][parent[i]];
    }
```

```c
    return mst_cost;
}

int main() {
    int V1, E1, V2, E2;

    printf("Enter number of vertices and edges for G1: ");
    scanf("%d %d", &V1, &E1);
    Edge* edges1 = (Edge*)malloc(E1 * sizeof(Edge));
    printf("Enter edge list for G1 (u v weight):\n");
    for (int i = 0; i < E1; i++) {
        scanf("%d %d %d", &edges1[i].u, &edges1[i].v, &edges1[i].weight);
    }

    printf("Enter number of vertices and edges for G2: ");
    scanf("%d %d", &V2, &E2);
    int** graph2 = (int**)malloc(V2 * sizeof(int*));
    for (int i = 0; i < V2; i++) graph2[i] = (int*)calloc(V2, sizeof(int));

    printf("Enter edge list for G2 (u v weight):\n");
    for (int i = 0; i < E2; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph2[u][v] = w;
        graph2[v][u] = w;
    }

    int c_u, c_v, c_w;
    printf("Enter the high-capacity transmission edge connecting G1 and G2 (u in G1, v in G2, weight): ");
    scanf("%d %d %d", &c_u, &c_v, &c_w);

    int cost1 = kruskalMST(edges1, V1, E1);
    int cost2 = primMST(graph2, V2);

    printf("\n--- Global Network MST ---\n");
    printf("Connecting Edge: G1(%d) - G2(%d) \tCost: %d\n", c_u, c_v, c_w);
    int total_global_cost = cost1 + cost2 + c_w;
    printf("\nTotal Cost of Global Spanning Tree: %d\n", total_global_cost);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> gcc .\Ques4.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6> ./a.exe
Enter number of vertices and edges for G1: 4 4
Enter edge list for G1 (u v weight):
0 1 2
1 2 3
2 3 1
0 3 4
Enter number of vertices and edges for G2: 3 3
Enter edge list for G2 (u v weight):
0 1 5
1 2 6
0 2 7
Enter the high-capacity transmission edge connecting G1 and G2 (u in G1, v in G2, weight): 3
0
20

--- G1 MST Edges (Kruskal) ---
2 - 3   Cost: 1
0 - 1   Cost: 2
1 - 2   Cost: 3

--- G2 MST Edges (Prim) ---
0 - 1   Cost: 5
1 - 2   Cost: 6

--- Global Network MST ---
Connecting Edge: G1(3) - G2(0)  Cost: 20

Total Cost of Global Spanning Tree: 37
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment6>
```

Figure 4: Output for Problem 4.